

Primary-Path Separation Manifests for Anonymous-DHT Receipts

Guard Objects for Non-Collusion Assumptions (Series Synthesis)

Unified archive note v0.05 (2026-02-27; archive v115)

Abstract

Many “reader-privacy” deployments rely on a primary path whose privacy story is not a pure numeric budget, but a *separation assumption*: one role sees client-side metadata while a different role sees destination-side metadata, and the claim is interpreted only if these roles do not collude within the threat window. This is a legitimate design axis (e.g., relay/gateway splits such as OHTTP[?]), but it is a common source of silent drift: the architecture name stays the same while the separation assumption quietly fails.

This note standardizes a small *separation manifest*: a digest-bound guard object that makes the non-collusion assumption, its invalidation triggers, and its evidence hooks explicit and diffable across releases. Other papers should cite this note instead of re-stating what “separation class `sep.nr1`” means.

Scope. This is an interface note. It does *not* claim that separation is sufficient for anonymity, and it does *not* provide a universal accountant that turns separation assumptions into bits. It only standardizes the guard object that makes such assumptions addressable, reviewable, and auditable under change control.

Imports. Receipt line items (including `obs_model` and `knobs`) are owned by Synthesis 12.[?] Trace-interface vocabulary (exposure normal forms) is owned by Synthesis 3.[?] Threat-window declarations are owned by Synthesis 4.[?] Replay hooks and drift/equivalence semantics are owned by Synthesis 18–20.[?, ?] User discovery/anchoring is owned by Synthesis 13 and Synthesis 9.[?, ?] A concrete instantiation lives in the worked example (Synthesis 17).[?]

Exports. A minimal object shape for *separation manifests* and a small editorial rule: *separation-class claims must name a manifest digest, and manifest drift is interface drift unless explicitly coarsened*. For deployments that want a small quantitative adjunct (separation failure as a receipt-grade $(0, \rho)$ checkpoint) without changing the receipt schema, see Synthesis 23.[?]

1 Why manifests (not prose)

A “separation” claim fails in practice when a deployment accidentally re-introduces a linking handle between the separated roles: shared keys, shared logs, co-resident operators, correlated identifiers, or a fallback that leaks destination information back to the client-side observer. These failures are rarely visible from the headline protocol name. A manifest makes them *enumerable* and therefore diffable.

2 Separation classes (names without semantics are not contracts)

In this archive, a separation class (e.g., `sep.nr1`) is a human mnemonic for a particular non-collusion assumption. The mnemonic is not a contract; the contract is a digest-bound manifest.

Definition 1 (Separation manifest (minimal)). *A separation manifest is a small, hash-addressable object containing:*

1. **Identity:** `sep_manifest_id` (digest id), plus a human label `sep_label` (e.g., `sep.nr1`).
2. **Context binding:** the `tw_id` (threat window) and the `exposure_nf_id` of the surface the manifest guards.[?, ?]
3. **Roles and partitions:** a list of roles (e.g., `client`, `relay`, `router`) and, for each role, a short description of which observation tiers/projections that role is assumed to see.
4. **Assignment / rotation policy:** how role instances are selected and rotated (e.g., per-request random relay selection from a declared set; per-epoch key rotation; explicit disjoint operator sets). This is the minimal hook needed to make separation claims replayable and (when desired) to justify a published separation-failure bound.[?]
5. **Non-collusion clause:** an explicit set of forbidden collusion edges (e.g., `relay` $\nleftrightarrow$ `router`) for the duration of the threat window.
6. **Linking handles:** the identifiers (keys, stable connection ids, log correlation ids) that would create linkability if shared across roles.
7. **Invalidation triggers:** a finite list of events/changes that cause the separation interpretation to become invalid (e.g., shared operator, shared key material, enabled debug logging, fallback policy change).
8. **Evidence hooks:** a replay-oriented pointer saying how an auditor would check the above (attestation type, log pointer, or a declared “policy-only” status).

Remark 1 (Why this is receipt-grade). *A manifest is intentionally small. It is not an evaluation trace; it is the guard surface that explains what a “0-bit” primary claim is actually asserting. It is therefore appropriate to treat it like a knob: commit to its digest and treat digest drift as claim drift.*

3 How to carry the manifest without changing the schema

Synthesis 12 deliberately keeps the public receipt schema minimal. Separation manifests therefore live *adjacent* to the receipt and are referenced by digest. Two standard placements are:

- **As a knob:** include `primary_surface_manifest_digest` inside the line item’s knobs.
- **As an observation qualifier:** include `sep_manifest_id` inside the line item’s `obs_model`.

Either choice is acceptable as long as (i) the manifest digest is committed, and (ii) the compare profile surfaces it for drift checks.[?, ?]

4 Drift rules (editorial)

A separation manifest is part of the interface claim. Accordingly:

- If `sep_manifest_id` changes, treat it as interface drift and issue a new receipt lineage (Certified C + Release A), unless the change is an explicit *coarsening* that preserves the guarded surface.[?]
- If the manifest invalidation triggers fire at runtime, the primary-path line item must be treated as out-of-scope (and any composite claim must fall back to the remaining budgeted surfaces).
- If a deployment wants quantitative primary-path budgets, use Regime II conditional certificates (Synthesis 21) and cite the spectral-to-budget translation (Anonymity D) for delegation-based designs.[?, ?]

5 Research warranted

This note standardizes the *guard object*; it does not solve the “how many bits does collusion probability buy me?” problem. Two promising directions are: (i) probabilistic separation (explicitly budgeting collusion risk within a window) and (ii) coupling guard manifests to measurable network-level independence evidence. Both would benefit from being packaged as replayable, digest-bound evidence objects compatible with Synthesis 21.

Synthesis 23 records the minimal first step: how a published upper bound on separation failure can be interpreted as a guarded $(0, \rho)$ checkpoint in the series’ existing endpoint semantics.[?]

References

- [1] Anonymous. Receipt Line Items for Anonymity Claims: A Minimal Tuple Schema for Published Budgets. Synthesis 12, The-One-True-Archive (2026).
- [2] Anonymous. Trace Interfaces for Anonymous DHTs. Synthesis 3, The-One-True-Archive (2026).
- [3] Anonymous. Threat Models and Repetition Windows for Anonymous DHTs. Synthesis 4, The-One-True-Archive (2026).
- [4] Anonymous. Digest-Bound Replay Plans and State Declarations: Equivalence, Minimality, and Change Control. Synthesis 18, The-One-True-Archive (2026).
- [5] Anonymous. Auditor Replay for Anonymous-DHT Receipt Line Items: A Mechanical Checklist for Digest-Bound Claims. Synthesis 20, The-One-True-Archive (2026).
- [6] Anonymous. User-Verifiable Receipt Interfaces for Anonymous-DHT Claims. Synthesis 13, The-One-True-Archive (2026).
- [7] Anonymous. Transparency Primitives for Privacy Receipts: Beacons, VRFs, and Append-Only Logs. Synthesis 9, The-One-True-Archive (2026).
- [8] Anonymous. Worked Example: Instantiating a Tiered Reader-Privacy Receipt. Synthesis 17, The-One-True-Archive (2026).
- [9] Anonymous. Probabilistic Separation Budgets: Separation Failure as a Receipt-Grade (ϵ, δ) Adjunct. Synthesis 23, The-One-True-Archive (2026).
- [10] Anonymous. Conditional Per-Step Budget Certificates: A Receipt-Grade Interface for Adaptive Horizon Claims. Synthesis 21, The-One-True-Archive (2026).

- [11] Anonymous. Spectral Delegation for Anonymous DHT Lookups: From mixing certificates to receipt-grade per-step budgets. Anonymity series Paper D, The-One-True-Archive (2026).
- [12] M. Thomson and C. A. Wood. *Oblivious HTTP*. RFC 9458, IETF, January 2024.